

INF-203 Project Report

Izza Qamar , Martin Braaten Kildal, Philip Adrian Kern Øvland

June 2026

Contents

1 Task 1	2
2 Task 2	2
3 Task 3	2
4 Task 4	2
5 Task 5	3
6 Task 6	3
7 Task 7	3
8 Task 8	3
9 Task 9	3
10 Task 10	3
11 Task 11	3
12 Task 12	3
13 Task 13	3
14 Task 14	3
15 Documentation of Generative AI Tools	3
16 Conclusion	4

1 Task 1

Our group created a shared GitHub repository for the project. The repository is used to store code and project files, allowing all members to contribute and track changes. This setup helps document each member's contributions and supports collaborative development throughout the course. The repository can be accessed here: [Inf-203 GitHub repository](#)

2 Task 2

We created a requirements register using a spreadsheet to manage the system requirements. The requirements are structured using an Agile approach with epics and user stories, and are prioritised using the MoSCoW method.

We defined two main epics: *Data Processing and Output* and *User Feedback and Support*. Each epic contains user stories describing system functionality from different user perspectives. The requirements include three personas: Student, Teaching Assistant, and Teacher. The requirements register will be maintained throughout the project and updated as the system is developed and refined. The requirements were maintained in a spreadsheet as shown below.

Requirements spreadsheet:

Label	Persona	Actionable Objective	Overarching Aim	Priority MoSCoW
Epic-1	Student	System accepts valid input data and displays correct output after processing	Data Processing and Output	
User story 1.1	Student	I want to see results so I can check correctness	Epic-1	Must
User story 1.2	Student	I want to input data so I can run the program	Epic-1	Must
Epic-2	Student, TA, Teacher	System shows error for invalid input and provides access to final results for evaluation	User Feedback and Support	
User story 2.1	Student	System shows error messages for invalid input	Epic-2	Should
User story 2.2	Teaching Assistant	I want to view outputs so I can help debug issues	Epic-2	Should
User story 2.3	Teacher	I want to review results so I can evaluate student submissions	Epic-2	Must

3 Task 3

For this task, we first set up three basic classes: Node, Edge, and Graph. Then we made each node store its own unique label along with lists of incoming and outgoing edges. After that, we created the Edge class so every edge keeps its label and knows which nodes it connects. Finally, we built the Graph class to manage everything, it keeps track of all nodes, makes sure labels are unique, and links edges into the right incoming/outgoing lists. Together, this gives us doubly linked incidence list graph that works for our knowledge-graph data.

4 Task 4

We created a function that loads a JSON-LD file, extracts the triples, and stores them in a list. The function then uses the list to create a graph using the triples, before it returns the graph. We ran a test run on the file: `linked-data-intro-context.json` and obtained the following output:

Additionally we created a quick program that visualizes the graph using the Kamada-Kawai layout. The graph is visualized with nodes representing subjects and objects, and edges representing predicates.

```

Nodes:
example-abox:ISSON25 Node(example-abox:ISSON25)
https://www.nanotextnology.com/index.php/isson/data# Node(https://www.nanotextnology.com/index.php/isson/data#)
https://schema.org/ Node(https://schema.org/)
schema:EducationEvent Node(schema:EducationEvent)
example-abox:Halliru_Ibrahim Node(example-abox:Halliru_Ibrahim)
schema:Person Node(schema:Person)
example-abox:Argiris_Laskarakis Node(example-abox:Argiris_Laskarakis)
example-abox:Stratos_Saliakas Node(example-abox:Stratos_Saliakas)
example-abox:NANOTEXNOLOGY_2025 Node(example-abox:NANOTEXNOLOGY_2025)
schema:BusinessEvent Node(schema:BusinessEvent)
example-abox:Stergios_Logothetidis Node(example-abox:Stergios_Logothetidis)

Edges:
Edge(example-abox, example-abox:ISSON25 -> https://www.nanotextnology.com/index.php/isson/data#)
Edge(schema, example-abox:ISSON25 -> https://schema.org/)
Edge(type, example-abox:ISSON25 -> schema:EducationEvent)
Edge(schema:performer, example-abox:ISSON25 -> example-abox:Halliru_Ibrahim)
Edge(type, example-abox:Halliru_Ibrahim -> schema:Person)
Edge(schema:performer, example-abox:ISSON25 -> example-abox:Argiris_Laskarakis)
Edge(type, example-abox:Argiris_Laskarakis -> schema:Person)
Edge(schema:performer, example-abox:ISSON25 -> example-abox:Stratos_Saliakas)
Edge(type, example-abox:Stratos_Saliakas -> schema:Person)
Edge(schema:superEvent, example-abox:ISSON25 -> example-abox:NANOTEXNOLOGY_2025)
Edge(type, example-abox:NANOTEXNOLOGY_2025 -> schema:BusinessEvent)
Edge(schema:organizer, example-abox:NANOTEXNOLOGY_2025 -> example-abox:Argiris_Laskarakis)
Edge(schema:organizer, example-abox:NANOTEXNOLOGY_2025 -> example-abox:Stergios_Logothetidis)
Edge(type, example-abox:Stergios_Logothetidis -> schema:Person)

Sample structure check:
Node: Node(example-abox:ISSON25)
Outgoing edges: [Edge(example-abox, example-abox:ISSON25 -> https://www.nanotextnology.com/index.php/isson/data#), Edge(schema, example-abox:ISSON25 -> https://s
chema.org/), Edge(type, example-abox:ISSON25 -> schema:EducationEvent), Edge(schema:performer, example-abox:ISSON25 -> example-abox:Halliru_Ibrahim), Edge(schem
a:performer, example-abox:ISSON25 -> example-abox:Argiris_Laskarakis), Edge(schema:performer, example-abox:ISSON25 -> example-abox:Stratos_Saliakas), Edge(schem
a:superEvent, example-abox:ISSON25 -> example-abox:NANOTEXNOLOGY_2025)]
Incoming edges: []

```

Figure 1: Graph output from JSON-LD parser test run

5 Task 5

6 Task 6

7 Task 7

8 Task 8

9 Task 9

10 Task 10

11 Task 11

12 Task 12

13 Task 13

14 Task 14

15 Documentation of Generative AI Tools

We used a Generative AI tool during this project to help set up our LaTeX report, with the basic structure and formatting.

We also used it to better understand the concept of knowledge graphs and how incidence list graph structures work, in particular how nodes store incoming and outgoing edges. This helped us when implementing the graph in Python.

16 Conclusion